

CL1002-Programming Fundamentals– FALL 2023

LAB 14

Pointers



Learning Outcomes

In this lab you are expected to learn the following:

- Pointers



Dynamic Memory Allocation in C++ for Arrays

Allocating a Block of Memory Using the "new" Operator

The **new** operator can also be used to allocate a block of memory (an array) of any `data_type`.

Syntax:

```
data_type* ptr_var = new data_type[size_of_the_array];
```

Example:

```
char* name = new char[10];
```

Standard Array Declaration vs Using the "new" Operator

The most significant distinction between standard **arrays** and a **dynamically allocated array** is that standard arrays are de-allocated by the online C++ compiler when the function execution finishes or it goes out of scope. While with dynamically allocated arrays, arrays always remain in the memory until either the programmer de-allocates them or the program execution finishes.

Syntax for normal array declaration:

```
int arr[5];
```

Syntax for dynamically allocated array:

```
int* arr = new int[5];
```



Example C++ Program to allocate memory using dynamic memory allocation:

```
#include <iostream>

using namespace std;

int main()
{
    // dynamically allocating an integer array of size 5 using new
    operator
    int* arr = new int[5];

    cout << "Enter 5 values in the array: ";

    // taking 5 values as an input from the user
    for(int i = 0; i < 5; i++)
    {
        cin >> arr[i];
    }

    // displaying the array elements in the output
    cout << "\nArray elements: ";
    for(int i = 0; i < 5; i++)
    {
        cout << arr[i] << " ";
    }

    // dynamically deallocating the array using delete operator
    delete [] arr;

    // garbage values will be displayed as
    // the pointer will point to some deallocated locations
    cout << "\nGarbage array values after deallocation of array memory: ";

    for(int i = 0; i < 5; i++)
    {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Sample Input:

```
Enter 5 values in the array: 1 2 3 4 5
```

Sample Output:

```
Array elements: 1 2 3 4 5
Garbage array values after deallocation of array memory: -1544938184 32704 -
1544938184 32704 5
```



Explanation:

- We have created a dynamically allocated array using the new operator.
- Then, we have taken input for the array elements from the user and printed the array elements.
- With the use of the delete operator on the array pointer arr, we have de-allocated the array memory.
- Now, the array location will contain garbage values as seen in the output.

Submission Instructions:

1. Create a single cpp file containing all the functions of the problems and main function.
2. Save the **cpp** file with the roll no and task number
e.g. i230001_Q1.cpp
3. Now create a new folder with *ROLLNO_LAB14_SEC* e.g. **i23XXXX_LAB14_A**
4. You need to display your roll no and name before the output of each question.
5. Now you have to submit this zipped file on Google Classroom.
6. If you don't follow the above-mentioned submission instructions, you will be marked **zero**.
7. Plagiarism in the Lab Task will result in **zero** marks in the whole category.



Lab Tasks

Problem 1

Write a program that concatenates two strings using pointers. The strings should be considered as character arrays. Take input strings from user.

Problem 2

Ask the user to input the size of the integer array. Declare an array of size entered by the user **dynamically** and input the values of the array. Now print the following:

- The number of positive integers.
- The number of negative integers.
- The number of odd integers.
- The number of even integers.

Problem 3

Code a C++ program to print 2 combinations of a given string; use **pointers** swapping. Use two iterations, in the first iteration, swap the first two characters and print the string variable. In the second iteration, swap the next two characters and print the string variable.

String: abcd

```
string mystring = "abcd";
```

you can access the characters of the string as:

```
char *p = &mystring[0];  
for(int i=0; i<4; i++){  
    cout<<*(p+i);  
}
```



example output:

1st combination: **bacd**

2nd combination: **badc**

Problem 4

Write a C++ function named **reverseArray** that takes an array of integers and its size as parameters. The function should dynamically allocate memory for the reversed array, reverse the order of the original array's elements, and return the reversed array.

Again, remember to free the dynamically allocated memory.

```
int* reverseArray(int* arr, int size) {  
  
    // Dynamically allocate memory for the reversed array  
  
    // Reverse the order of array elements  
  
    // Free the dynamically allocated memory for the original array  
  
    // Return the reversed array  
  
}
```

Problem 5

Write a C++ program that takes a string as input and counts the number of vowels (a, e, i, o, u) and consonants in the string using pointers. Implement a function that takes a string and two integer pointers as parameters. The function should update the values pointed to by the pointers to store the count of vowels and non-vowels in the string



Example

Input: I like programming

Number of vowels: 5

Number of consonants: 11

Problem 6

Write a C++ function named **mergeArrays** that takes two arrays of integers and their respective sizes as parameters. The function should dynamically allocate memory for a new array, merge the elements of the two arrays, and return the merged array. Once again, ensure that you free the dynamically allocated memory.

```
int* mergeArrays(const int* arr1, int size1, const int* arr2, int size2) {  
  
    // Your code here  
  
    // Dynamically allocate memory for the merged array  
  
    // Merge the elements of the two arrays into the new array  
  
    // Return the merged array  
  
}
```